

Chapter 5: JavaScript

1 History

JavaScript, invented by Brendan Eich and developed by Netscape, first appeared in **1995** under the name **LiveScript**, with the goal of making web pages interactive. Its usage quickly spread, and it was soon adopted by other browsers. Today, it is widely used on websites.

2 Definition

JavaScript (JS) is a lightweight, object-oriented scripting language, primarily known as the scripting language for web pages. JavaScript code is **interpreted** or **just-in-time (JIT) compiled**. It is a **prototype-based** language with **weak and dynamic typing**.

Just as the **W3C** defines standards for (X)HTML, the standard for JavaScript is **ECMAScript**. As of **2012**, all modern browsers fully support **ECMAScript 5.1**.

3 Characteristics of JavaScript

- **It is a script:** Unlike compiled languages, scripts are interpreted.
- **Executed by the client's browser:** Its behavior depends on the browser.
- **Basic syntax is similar to Java and C++.**
- **Prototype-based object-oriented language:** Objects are created from prototypes.
- **Dynamically and weakly typed.**
- **Relatively limited:** Mostly confined to the web page where it runs.
- **Not compiled:**
 - Faster to write (no compiler needed, single file, ...).
 - Less powerful than, for example, a C program.

4 Usage

Its primary use is within web pages, making them **interactive**. Examples of what it can do:

- Open pop-ups.
- Scroll text.
- Insert dynamic menus (expand on hover).
- Create slideshows (image rotation, pause buttons, navigation, etc.).

5 DHTML

DHTML (Dynamic HTML) refers to pages where content (or layout) changes **without reloading**. This term is not officially defined by the W3C.

6 Methods of Inserting JavaScript

6.1 Directly in HTML Tags

6.1.1 Using event handler

- Code is written inside the relevant tag using **event handlers** (e.g., **onclick**).
- **Example:**

```
<a href="#" onclick="alert('Hello!');">Link</a>
```

- Other common event handlers:
 - **onclick** (left mouse click)
 - **ondblclick** (double-click)
 - **onmouseover** (mouse hover)
 - **onmouseout** (mouse leaves element)
 - **onload** (page load)
 - **onunload** (page exit)
- **Example:** Event Handlers in the <body> Tag

```
<body onload="alert('Hello!');" onunload="alert('Goodbye!');">  
  <!-- page content -->  
</body>
```

1. **onload**

- Triggers when the page **finishes loading** (HTML, CSS, images, etc.)
- Displays "Hello!" in a popup

2. **onunload**

- Triggers when the user **leaves the page** (closing/navigating away)
- Displays "Goodbye!" (*may be blocked by some browsers*)

6.1.2 JavaScript in Links

You can write JavaScript directly in a link's **href** by prefixing it with **javascript:**, like this:

```
<a href="javascript:alert('Hello!');">Click here</a>
```

✓ **Prevents page scroll to top** (unlike # in links).

✓ **Quick inline scripts** without needing event handlers.

➤ **How It Works:**

- When clicked, the browser executes the JavaScript code instead of navigating.
- **No page reload** or unwanted jump to the top (common with href="#").

6.2 Between <script> Tags

```
<script type="text/javascript">
// JavaScript code here
</script>
```

- **Place scripts in the <body>** for execution during page load.
- **Place in <head>** for deferred execution (e.g., triggered by events).
- **Legacy browser support:**

```
<script type="text/javascript">
<!--
// Code hidden from old browsers
//-->
</script>
```

6.3 External File Import

- Save JavaScript in a **.js** file and link it in the head tag:

```
<script type="text/javascript" src="script.js"></script>
```

7 JavaScript Syntax

7.1 Statements:

- Separated by line breaks or semicolons (;).
- Semicolons are optional if statements are on separate lines.
- JavaScript is **whitespace-insensitive**.

7.2 Comments:

- Single-line: `// Comment`
- Multi-line: `/* Comment */`

7.3 Functions:

- Defined as `functionName();` (with or without parameters).
- **Example:** `alert('Hello world');`

7.4 Objects:

- JavaScript is **object-oriented**.

- Objects have:
 - **Constructor:** Code executed when creating an object.
 - **Properties:** Variables holding object data.
 - **Methods:** Functions modifying the object.
- **Example:**

```
var myString = 'Hello';  
alert(myString.length); // Property  
alert(myString.toUpperCase()); // Method
```

7.5 Variables:

- Declared with var (e.g., var x;).
- **Dynamic typing:** No need to specify type (unlike C).
- **Primitive types:**
 - number (e.g., var x = 5;)
 - string (e.g., var text = 'Hi';)
 - boolean (e.g., var flag = true;)
- **Type checking:** typeof variable (returns "undefined" if nonexistent).
- **Naming rules:**
 - Can include letters, digits, _.
 - Cannot start with a digit.
 - Case-sensitive (myVar ≠ myvar).
 - Avoid reserved keywords (new, delete, etc.).

7.6 Strings:

- Delimited by " or '.
- Escape characters: \", \\, etc.
- Concatenation: + (e.g., "Hello " + name).

7.7 Numbers:

- Use “.” for decimals (not “,”).
- Division by zero returns Infinity or -Infinity.
- Operators:
 - +=, -=, *=, /=, %=
 - ++, -- (increment/decrement)

7.8 Important Notes:

- **Do not declare the same variable twice.**
- **Strings** must be enclosed in:
 - Double quotes ("Hello")
 - Single quotes ('Hello')
 - **Inside HTML tags, prefer single quotes** (since HTML uses " for attributes).
- **Escaping characters:**
 - Use \" for quotes inside strings:

```
let text = 'He said, \"Hello\"';
```

- Use \\ to display a backslash.
- **Concatenation** uses +:


```
let fullName = "Mohamed" + " " + "Ibrahim";
```
- **Numbers:**
 - Use “.” for decimals (not “,”).
 - Division by zero returns Infinity or -Infinity.
- **Assignment shortcuts:**

Shortcut	Equivalent
x += y	x = x + y
x -= y	x = x - y
x *= y	x = x * y
x /= y	x = x / y

Shortcut	Equivalent
x %= y	x = x % y
x++	x += 1
x--	x -= 1

7.9 Examples:

```
// Strings & Quotes
let msg1 = "Hello";    // Double quotes
let msg2 = 'World';    // Single quotes (preferred in HTML)
let quote = 'She said, \"Hi!\"'; // Escaping inner quotes

// Numbers
let price = 9.99;      // Decimal point (not comma)
let infinityCheck = 10 / 0; // Returns "Infinity"

// Operators
let count = 5;
count += 3; // 8 (same as count = count + 3)
count--;   // 7 (decrement)
```

7.10 Conditionals

7.10.1 Comparison operators:

Operator	Meaning
==	Equal to
!=	Not equal
===	Strict equal (type + value)
!==	Strict not equal
>, <	Greater/Less than
>=, <=	Greater/Less than or equal

➤ **How === and !== Work (Strict Equality/Inequality)?**

- **== (Loose Equality)** → Checks **value only** (converts types if needed).
- **=== (Strict Equality)** → Checks **both value AND type** (no conversion).

Example:

```
var number = 4;    // Number
var text = '4';    // String
var result;

// Loose Equality (==)
result = number == text;
alert(result);    // true (converts string '4' to number 4)

// Strict Equality (===)
result = number === text;
alert(result);    // false (different types: number ≠ string)
```

Operator	Use Case	Example
==	When type conversion is acceptable	5 == '5' → true
===	When exact type matching is required	5 === '5' → false

Best Practice:

✓ **Prefer ===** to avoid unexpected type coercion bugs.

Common Pitfalls:

```
0 == false    // true (0 and false are "falsy")
0 === false   // false (number ≠ boolean)
```

```
null == undefined // true (special JS rule)
null === undefined // false (different types)
```

Rule of Thumb:

"Use === unless you explicitly need type coercion."

7.10.2 Logical Operators:

- && (AND), || (OR), ! (NOT)

7.10.3 Conditional Statements:

- **if...else if...else:**

```
if (condition1) {
  // Code
} else if (condition2) {
  // Code
} else {
  // Default
}
```

- **switch:**

```
switch (exp) {
  case 1: /* Code */ break;
  case 2: /* Code */ break;
  default: /* Code */
}
```

- **Ternary Operator:**

Example:

```
const message = adult ? "18+" : "-18";
// Equivalent to:
// if (adult) { message = "18+"; } else { message = "-18"; }
```

If the variable **adult** is true, the ternary operator returns the value **immediately after the question mark (?)**.

If adult is false, it returns the value **after the colon (:)**.

7.10.4 Loops

- **while:**

```
while (condition) { /* Code */ }
```

- **do...while:**

```
do { /* Code */ } while (condition);
```

- **for:**

```
for (var i = 0; i < 5; i++) { /* Code */ }
```

7.11 Arrays

- **Declaration:**

```
var myArray = [31, "Ali", 42, "Dalila"];
```

- **Accessing Elements:**

```
alert(myArray[1]); // "Ali"
```